Project 2: Food Delivery App — Student Guide

The question: Why are repeat customers leaving in certain cities?

This time **you lead**. There's no walkthrough. Use the same four phases you used in Project 1 (Retail), and use your retail files as a template.

Some things to know before you start:

- There is **more than one cause**.
- There is at least **one tempting explanation that is wrong**.
- No single query gives you the answer. You find it by testing ideas one at a time.

Contents

1. What you will hand in
2. Before you start
3. Words you will see
4. Meet the data
5. Definitions to use
6. Phase 1: SQL
7. Phase 2: Excel
8. Phase 3: Python
9. Phase 4: Power BI
10. Phase 5: The verdict
11. Checkpoints and grading
12. Common mistakes
13. SQL Server 2016 cheat sheet
14. When you get stuck

---

1. What you will hand in

#	Deliverable	Made in
1	A `.sql` file with commented queries and row-count checks	SQL portal or SSMS
2	An Excel workbook: audit sheet, issue log, pivots, hypotheses	Excel
3	A notebook (`.ipynb`): cleaning, checks, hypothesis tests, CSV export	Colab or Jupyter
4	Star-schema CSV files	Python
5	A Power BI report, 3 pages plus an About page: a `.pbix` file, or a report link plus PDF if you use the browser	Power BI
6	A one-page verdict	Word or PDF

The one-page verdict matters most. If your manager reads only that page, they should know why customers are leaving and what to do.

---

2. Before you start

Choose your tools

Use the same tools you chose for Project 1. The Project 1 guide (section 2) has the full setup steps for each one.

Phase	Option A	Option B
SQL	**Online SQL portal** at `ssms.penguins.digital`	**SSMS** on Windows
Excel	**Excel for the web** or Microsoft 365	**An older Excel**, such as 2016 or 2019
Python	**Google Colab** in the browser	**Jupyter** on your own computer (pandas 2.0 or later)
Power BI	**Power BI Desktop** on Windows	**Power BI in the browser** on a Mac

Quick reminders:

- **Portal:** it runs everything in the query tab and shows only the first result, so keep one query per tab. The grid shows up to 500 rows; **Export CSV** saves every row. Queries time out after 60 seconds.
- **Older Excel:** no `UNIQUE` , `XLOOKUP` or `FILTER` . Use **Remove Duplicates** or a PivotTable for distinct values, and `INDEX` / `MATCH` for lookups.
- **Colab:** put `capstone_food/` in Google Drive and start your notebook with:

```
from google.colab import drive
drive.mount("/content/drive")
%cd /content/drive/MyDrive/capstone_food
```

- **Jupyter:** `pip install pandas numpy openpyxl jupyter`
- **Power BI on a Mac:** use a Windows lab PC, or Power BI in the browser at `app.powerbi.com` with your school account. In the browser, upload your star schema as one Excel workbook, and hand in a report link plus a PDF instead of a `.pbix` file.

Set up your folder

```
capstone_food/
    food_queries.sql
    food_audit.xlsx
    food_analysis.ipynb
    csv/
    food_report.pbix
    food_verdict.docx
```

Test your connection

```
SELECT COUNT(*) AS order_count FROM dbo.food_orders;
```

You should see **25,366**.

---

3. Words you will see

If a word is new, check the glossary in the Project 1 guide too. These ones matter most here:

Word	Plain meaning
Grain	What one row means. In `food_orders`, one row is one order. In `food_order_items`, one row is one menu item on one order.
Repeat customer	A customer who orders again. We measure it in a 90-day window (see the definitions).
Retention / churn	Retention is customers who stay. Churn is customers who leave. They are two sides of the same coin.
Cohort	A group of customers who started at the same time, such as everyone who signed up in March 2024.
Right censoring	When you can't see the full outcome yet. A customer who ordered on December 28 hasn't had 90 days to come back, so you can't tell if they will.
Feed table	A messy import. No keys, everything stored as text. Clean it before you trust it.
Window function	A calculation that looks at other rows without collapsing them, such as `LAG`, `LEAD` or `ROW_NUMBER`. Very useful for "what did this customer do next?"
Rate vs count	A count is "how many." A rate is "how many out of how many." Compare cities with rates, because cities are different sizes.

---

4. Meet the data

A US food delivery app in 6 cities, from January 2024 to December 2025.

Table	Rows	One row is	Type
`food_cities`	6	A city. `province` holds the US state.	Core
`food_zones`	40	A delivery zone inside a city	Core
`food_restaurants`	800	A restaurant partner	Core
`food_menu_items`	6,000	A menu item at one restaurant	Core
`food_menu_price_history`	2,500	One price change on one menu item	Core
`food_customers`	4,000	An app customer	Core
`food_riders`	150	A delivery rider	Core
`food_orders`	25,366	An order (the header)	Core
`food_order_items`	60,727	**One menu item on one order**	Core
`food_deliveries`	25,366	The delivery timings for one order	Core
`food_ratings_feed`	13,065	A customer rating from an outside system	**Feed**

How the tables connect

```
food_cities ──< food_zones
     │  └──< food_restaurants ──< food_menu_items ──< food_menu_price_history
     │  └──< food_customers
     │  └──< food_riders
     │
food_customers   ──< food_orders >── food_restaurants
food_riders      ──< food_orders         (rider_id can be NULL)
food_orders      ──< food_order_items >── food_menu_items
food_orders      ─── food_deliveries    (exactly one per order)

food_ratings_feed ·····> food_orders.order_number   (text match, not a key)
```

To get from	To	Join on
`food_orders`	`food_customers`	`customer_id`
`food_orders`	`food_restaurants`	`restaurant_id`
`food_orders`	`food_riders`	`rider_id` (**NULL** for some cancelled or failed orders)
`food_orders`	`food_deliveries`	`order_id` (one to one)
`food_order_items`	`food_orders`	`order_id`
`food_order_items`	`food_menu_items`	`item_id`
`food_menu_items`	`food_restaurants`	`restaurant_id`
`food_menu_price_history`	`food_menu_items`	`item_id`
`food_restaurants`, `food_customers`, `food_riders`	`food_cities`	`city_id`
`food_ratings_feed`	`food_orders`	`order_ref_text` = `order_number`, after cleaning

The full list of columns is in the food data dictionary (`capstone/docs/food_data_dictionary.md`).

Things that trip people up

- **Two kinds of "city."** A customer has a home city (`food_customers.city_id`), and a restaurant has its own city (`food_restaurants.city_id`). Decide which one each question needs, and write it down.
- The order total isn't stored as one number. `subtotal` , `delivery_fee` and `discount_amount` are separate columns.
- `food_orders` includes **Cancelled** and **Failed** orders. Delivery times only make sense for **Delivered** ones.
- Menu item names repeat across restaurants. Use `item_code` or `item_id` to identify one item.
- `distance_km` is in kilometers. Multiply by 0.621 for miles if you want to show miles.

- Percentages are stored as **fractions**: `0.1250` means 12.5%.

---

5. Definitions to use

Everyone in class uses these, so everyone's numbers can be compared. Write them at the top of your `.sql` file.

Metric	Definition
Delivered order	`order_status = 'Delivered'`
Actual minutes	`DATEDIFF(MINUTE, food_orders.order_datetime, food_deliveries.delivered_at)`
Minutes late	Actual minutes minus `promised_minutes`
Late delivery	Minutes late is **over 10**
90-day repeat rate	Of customers with a delivered order in a quarter, the share who order again within 90 days of their **last** order in that quarter
Comparison period	Q2 and Q3 of 2024 against Q2 and Q3 of 2025

- **Don't use Q4 2025 for repeat rate.** Customers who ordered in December haven't had 90 days to come back yet.
- Show the group size (how many customers or orders) next to every rate.

---

6. Phase 1: SQL: build a trusted extract

Goal: confirm the headline, understand the tables, and save clean inputs for Excel and Python.

Step 1: Count the rows

```
SELECT 'orders' AS table_name, COUNT(*) AS row_count FROM dbo.food_orders
UNION ALL SELECT 'order_items', COUNT(*) FROM dbo.food_order_items
UNION ALL SELECT 'deliveries', COUNT(*) FROM dbo.food_deliveries
UNION ALL SELECT 'customers', COUNT(*) FROM dbo.food_customers
UNION ALL SELECT 'riders', COUNT(*) FROM dbo.food_riders
UNION ALL SELECT 'ratings_feed', COUNT(*) FROM dbo.food_ratings_feed;
```

✅ **Check:** your counts match the table in section 4.

Step 2: Look at the status column

```
SELECT order_status, COUNT(*) AS orders
FROM dbo.food_orders
GROUP BY order_status
ORDER BY orders DESC;
```

✅ **Check:** Delivered 23,564, Cancelled 1,277 and Failed 525.

Step 3: Delivered orders by city and year

This uses the **restaurant's** city.

```
SELECT c.city_name,
       c.province AS state,
       YEAR(f.order_datetime) AS order_year,
       COUNT(*) AS delivered_orders
FROM dbo.food_orders AS f
JOIN dbo.food_restaurants AS r ON r.restaurant_id = f.restaurant_id
JOIN dbo.food_cities AS c ON c.city_id = r.city_id
WHERE f.order_status = 'Delivered'
GROUP BY c.city_name, c.province, YEAR(f.order_datetime)
ORDER BY c.city_name, order_year;
```

✅ **Check:** 6 cities, 2 years each, 12 rows. For example, New York has 2,918 delivered orders in 2024.

Notice: orders are growing in every city. So why would customers be leaving? Keep that question in mind.

Step 4: Work out lateness for each order

```
SELECT TOP 20
       f.order_number,
       f.order_datetime,
       d.promised_minutes,
       DATEDIFF(MINUTE, f.order_datetime, d.delivered_at) AS actual_minutes,
       DATEDIFF(MINUTE, f.order_datetime, d.delivered_at) - d.promised_minutes AS
minutes_late,
       CASE WHEN DATEDIFF(MINUTE, f.order_datetime, d.delivered_at) - d.promised_minutes >
10
            THEN 1 ELSE 0 END AS is_late
FROM dbo.food_orders AS f
JOIN dbo.food_deliveries AS d ON d.order_id = f.order_id
WHERE f.order_status = 'Delivered'
ORDER BY f.order_id;
```

✅ **Check:** order `FD-0000004` was promised in 37 minutes, took 54, and is late (`is_late = 1`).

Tip: a 1/0 flag is handy, because `AVG(1.0 * is_late)` gives you the late **rate**.

Step 5: The headline: 90-day repeat rate

This is the hardest query in the project, so it's given to you. Read it one step at a time.

```
-- Step A: one row per delivered order, with just the date
WITH delivered AS (
    SELECT customer_id, CAST(order_datetime AS DATE) AS order_day
    FROM dbo.food_orders
    WHERE order_status = 'Delivered'
),
-- Step B: one row per customer per quarter, with their last order in that quarter
customer_quarters AS (
    SELECT customer_id,
           YEAR(order_day) AS order_year,
           DATEPART(QUARTER, order_day) AS order_quarter,
           MAX(order_day) AS last_order_day
    FROM delivered
    GROUP BY customer_id, YEAR(order_day), DATEPART(QUARTER, order_day)
),
-- Step C: did they order again within 90 days of that last order?
flagged AS (
    SELECT q.*,
           CASE WHEN EXISTS (SELECT 1 FROM delivered AS d
                             WHERE d.customer_id = q.customer_id
                               AND d.order_day > q.last_order_day
                               AND d.order_day <= DATEADD(DAY, 90, q.last_order_day))
                THEN 1 ELSE 0 END AS came_back
    FROM customer_quarters AS q
)
-- Step D: the rate, for Q2 and Q3 only
SELECT order_year,
       COUNT(*) AS customer_quarters,
       SUM(came_back) AS came_back,
       CAST(1.0 * SUM(came_back) / COUNT(*) AS DECIMAL(6,4)) AS repeat_rate_90d
FROM flagged
WHERE order_quarter IN (2, 3)
GROUP BY order_year
ORDER BY order_year;
```

✅ **Check:** about **0.6223** in 2024 and **0.5917** in 2025, so the repeat rate fell from about 62% to about 59%.

Your turn: the question says "in **certain** cities." Change this query so it shows the repeat rate by **customer home city** and year. Hint: add `city_id` in Step A by joining to `food_customers`, then carry it through each step.

Step 6: Look at the ratings feed before joining it

```
SELECT TOP 50 *
FROM dbo.food_ratings_feed
ORDER BY NEWID();
```

```
SELECT rating_text, COUNT(*) AS rows
FROM dbo.food_ratings_feed
GROUP BY rating_text
ORDER BY rows DESC;
```

Look for:

- Ratings written as `4`, `4.0`, `4 stars`, `four` or `4/5`. They all mean the same thing.
- Order references with and without a dash, in lower case, or with spaces
- Dates in several formats
- The same `rating_ref` appearing more than once
- Blank rows and `N/A` values
- Ratings whose order doesn't exist

Step 7: Save your extracts

Decide the grain of each extract **before** you write the query.

Extract	Suggested grain	Ideas for columns
Orders	One row per order	Order, date and time, status, customer, customer city, restaurant, restaurant city, cuisine, rider, money columns, delivery times, minutes late
Customers	One row per customer	Every column, plus first order date and number of delivered orders
Restaurants	One row per restaurant	Every column, plus city name
Riders	One row per rider	Every column, plus city name
Ratings feed	One row per feed row, untouched	Every column, as is
Price history	One row per price change	Item, restaurant, city, date, old and new price

Saving extracts: in the portal, click **Export CSV** (it saves every row). In SSMS, right-click the grid and choose **Save Results As...**. Save into your `csv/` folder, in Google Drive if you use Colab.

✅ **Check:** your orders extract has exactly 25,366 rows. If it has more, a join is duplicating rows.

7. Phase 2: Excel: audit and explore

Audit first

Careful with the ratings feed in Excel. Excel can turn ratings such as `4/5` into dates (April 5) when it opens a CSV. In desktop Excel, use **Data > From Text/CSV**, choose **Transform Data**, set the feed columns to **Text**, then load. In Excel for the web, check a few cells against the SQL grid; if values were changed, audit that column in Python instead.

Look for problems in each extract and log them. Some places to look:

- **Ratings feed:** mixed rating formats, duplicates, blanks, `N/A`, orphans (ratings for orders that don't exist)
- **Orders:** cancelled or failed orders with no `cancel_reason`
- **Menu prices:** prices that look like typos (far too high or too low for that item)
- **Deliveries:** timings that don't make sense

Issue log

#	Table	Column	Problem	How many rows	What I will do
1	ratings_feed	rating_text	Same rating in 5 formats	?	Convert all to a number 1 to 5
2	...	...	...	...	...

Every issue needs a **count** and a **decision**.

Explore with pivots

Start broad, then narrow down:

1. **The headline:** repeat rate by city, 2024 against 2025. Which cities dropped?
2. **Lateness:** late rate by city and by month or quarter. When did it change?
3. **Time of day:** late rate by hour of the day. Are some hours worse?
4. **Ratings:** average rating by city and by restaurant (after cleaning). Are some restaurants rated far lower?
5. **Prices:** price changes by city. Did prices rise faster somewhere?

Write your hypotheses

Write **at least 4**, and make them specific. Angles to consider:

- **Service:** lateness, cancellations, failed deliveries, time of day, day of week
- **Supply:** riders and restaurants
- **Price:** menu price rises, delivery fees, discounts and vouchers
- **Customers:** acquisition channel, age band, device, new or existing customers
- **Place:** city and zone

Include the obvious suspects. Part of your grade is showing which explanations are wrong.

---

8. Phase 3: Python: clean and test

Load and clean

```
import pandas as pd

orders = pd.read_csv("csv/orders.csv", parse_dates=["order_datetime", "delivered_at"])
ratings = pd.read_csv("csv/ratings_feed.csv", dtype=str)   # keep the feed as text for now

print(orders.shape, ratings.shape)
```

Cleaning the ratings, step by step:

1. Drop rows where everything is blank.
2. Clean `rating_ref` (strip spaces, upper case), then drop duplicates.
3. Clean `order_ref_text` the same way you cleaned order numbers in Retail, so it matches `order_number`.
4. Turn `rating_text` into a number from 1 to 5. Lower-case it, then handle each format: digits, `.0`, `stars`, `/5` and words such as `four`.
5. Parse `submitted_at_text` into a date with `pd.to_datetime(..., format="mixed", errors="coerce")`.
6. Find orphans with a left merge to orders and `indicator=True`.

```
words = {"one": 1, "two": 2, "three": 3, "four": 4, "five": 5}
# Your code: build a clean rating column from rating_text
```

Prove it worked:

```
assert ratings["rating"].dropna().between(1, 5).all()
assert ratings["rating_ref"].is_unique
```

Useful pandas for this project

Task	pandas tool
Each customer's next order	`df.sort_values(["customer_id", "order_datetime"])` then `groupby("customer_id")["order_datetime"].shift(-1)`
Running count of late orders per customer	`groupby("customer_id")["is_late"].cumsum()`
Each customer's first order	`groupby("customer_id").head(1)` after sorting
Rate by group	`groupby("city")["is_late"].mean()`
Group sizes too	`groupby("city")["is_late"].agg(["mean", "size"])`

Test each hypothesis

For every hypothesis, write:

1. The hypothesis
2. The code that tests it
3. The result, **with group sizes**
4. **Supported** or **ruled out**, and why

Questions worth testing:

- Does a customer's experience on one order change whether they come back? Try comparing what happens after on-time and late orders.
- Do new customers and long-time customers behave differently?
- If something looks worse, **when** did it start? Does the timing match the drop in repeat rate?
- If one city looks unusual on one measure, does that city's repeat rate actually fall?

Size it

How many customers would have come back if each cause hadn't happened? If you can, turn that into orders or dollars.

Export a star schema

File	Type	One row is
`fact_orders.csv`	Fact	An order, with delivery timings and a late flag
`fact_ratings.csv`	Fact	A cleaned, de-duplicated rating
`dim_customer.csv`	Dimension	A customer
`dim_restaurant.csv`	Dimension	A restaurant
`dim_city.csv`	Dimension	A city
`dim_date.csv`	Dimension	A calendar day

You can add more, such as `dim_rider`, if your analysis needs them.

---

9. Phase 4: Power BI: model and report

1. **Load** your star-schema CSVs and connect each fact to each dimension, one to many. In the browser on a Mac, upload them as one Excel workbook instead, with one sheet per table (`df.to_excel` with `pd.ExcelWriter`, as in the Project 1 guide).
2. **Measures:** Delivered Orders, Late Rate, Average Rating and 90-Day Repeat Rate. For the repeat rate, it's fine to calculate a `came_back` flag in Python and average it in DAX.

   ```
   Late Rate = DIVIDE(SUM(fact_orders[is_late]), COUNTROWS(fact_orders))
   ```

3. **Reconcile:** check that repeat rate and late rate by city match your Python numbers **before** you build charts.
4. **Build 3 pages plus About:**
   - Page 1: the executive summary. Which cities, how much, and why.
   - Pages 2 and 3: one page for each cause, with the evidence.
   - About: sources, definitions, cleaning steps and limits.

Report checklist

- [ ] Chart titles state the finding, not just the measure
- [ ] One highlight color for the problem, gray for everything else
- [ ] Every rate shows its group size
- [ ] Q4 2025 is excluded from repeat rate, and the About page says why

---

10. Phase 5: Write the verdict

One page. Answer first.

Section	What to write
Answer	One or two sentences. Why are repeat customers leaving, and where?
Evidence	The two or three numbers that prove it.
Size	How many customers (or dollars) each cause cost.
Ruled out	The tempting explanations that turned out to be wrong, and why.
Action	Specific next steps, with an owner.
Risks and limits	What you couldn't check, and what could change your answer.

---

11. Checkpoints and grading

Checkpoints

#	Show your instructor
1	Your metric definitions and extract row counts
2	Your issue log and at least 4 written hypotheses
3	Clean numbers next to raw numbers, with the differences explained
4	A Power BI table that matches your Python numbers
5	A draft verdict, for peer review

Grading (100 points)

Area	Points	Excellent looks like
SQL	20	Correct grain, right joins, row checks, readable CTEs
Excel audit	15	Every issue counted, with a decision for each
Python	20	Repeatable, with asserts; hypotheses tested and some ruled out
Power BI	20	A sound star model, reconciled measures, findings as titles
Root cause	15	All causes found, sized, wrong theories rejected
Verdict	10	Answer first, specific actions, honest limits

---

12. Common mistakes

- Measuring lateness on cancelled or failed orders
- Using Q4 2025 for repeat rate
- Joining the ratings feed before cleaning and de-duplicating it
- Mixing up customer city and restaurant city without saying which one you used
- Averaging percentages instead of dividing totals
- Comparing counts when rates are the right measure
- Trusting a rate from a tiny group, such as one restaurant with three ratings
- Stopping at the first interesting pattern. There's more than one cause.
- Reporting without a size or an action

Two analyst habits worth building

1. **Big isn't the same as bad.** The city or restaurant with the most of something is often just the biggest. Compare rates.
2. **Something can be true and still not be the cause.** If a measure looks bad in one city, check whether that city's customers are actually leaving. A real pattern that doesn't line up with the drop isn't your

answer.

13. SQL Server 2016 cheat sheet

Need	Use
Trim spaces	`LTRIM(RTRIM(x))`. `TRIM` doesn't exist in 2016.
Text to number safely	`TRY_CONVERT(DECIMAL(12,2), x)`
Text to date (MM/DD/YYYY)	`TRY_CONVERT(DATE, x, 101)`
Text to date (M/D/YY)	`TRY_CONVERT(DATE, x, 1)`
Text to date (YYYY-MM-DD)	`TRY_CONVERT(DATE, x, 23)`
Text to date (Mar 1 2025)	`TRY_CONVERT(DATE, x, 100)`
Hour of the day	`DATEPART(HOUR, order_datetime)`
Day of the week	`DATENAME(WEEKDAY, order_datetime)`
Quarter	`DATEPART(QUARTER, d)`
Next row for the same customer	`LEAD(order_datetime) OVER (PARTITION BY customer_id ORDER BY order_datetime)`
Number each customer's orders	`ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_datetime)`
Running total	`SUM(is_late) OVER (PARTITION BY customer_id ORDER BY order_datetime ROWS UNBOUNDED PRECEDING)`
Active on a date	`start_date <= @d AND (end_date IS NULL OR end_date >= @d)`
Add to a date	`DATEADD(DAY, 90, d)`
Avoid integer division	`1.0 * a / b`
Random sample	`SELECT TOP 50 * FROM t ORDER BY NEWID()`
Remove duplicates	`ROW_NUMBER() OVER (PARTITION BY key ORDER BY ...)` and keep `= 1`

Note: you can't put `EXISTS` inside `SUM()` or `AVG()`. Make the flag in a CTE first, as in Step 5, then add it up.

14. When you get stuck

1. **Re-read the definitions.** Especially the 90-day window and the quarters to compare.
2. **Count rows** before and after every join.
3. **Follow one customer.** Pick one `customer_id` and list all their orders in date order. Does your logic give the right answer for them?

4. **Make it smaller.** Test on one city or one month first.
5. **Check the data dictionary.** It says what can be NULL and why.
6. **Ask a classmate about approach.** Each person still submits their own work.
7. **Ask your instructor.** Bring the query, what you expected, and what you got.